

Project 1: Linked Sequence Data Structure

Learning Objectives

By completing this project, you will:

- Design and implement a reusable container class in C++
 - Encapsulate linked list mechanics behind a clean public interface
 - Correctly manage dynamic memory ownership inside a class
 - Build on node-level list mechanics developed in Assignment 8
-

Overview

In Assignment 8, you implemented the **mechanics** of a singly linked list by working directly with nodes and pointers.

In this project, you will take the next step by implementing a **reusable container class** called `Sequence` that delegates low-level operations to a linked list implementation.

The internal linked list implementation will be hidden behind a fixed public interface.

This separation mirrors how real container classes are built.

Required Files

Your solution must include the following files:

- `Sequence.h`
- `Sequence.cpp`
- `linked_list.h`

You will also be given a test harness.

Relationship to Assignment 8

You must reuse and extend your linked list implementation from Assignment 8.

- All **node-level logic** must remain in `linked_list.h`
 - The `Sequence` class must **not** manipulate node pointers directly
 - `Sequence` is responsible for:
 - allocating nodes
 - managing ownership and lifetime
 - enforcing bounds and preconditions
 - exposing a clean public interface
-

Element Type

The `Sequence` class stores elements of type `std::string`.

To support this feature, the class defines a type alias:

```
using ElementType = std::string;
```

All public member functions must use `ElementType` rather than `std::string` directly.

The linked list implementation must **not** define or assume a concrete element type.

It operates on nodes that store `ElementType`, which is defined by `Sequence`.

Linked List Interface (Internal)

Your `linked_list.h` file **must** provide the following interface.

The interface is not public-facing and exist only to support **Sequence**

```
// linked_list.h

#pragma once

// ElementType must be defined before including this file

struct Node {
    ElementType value;
    Node* next;
};

void push_front(Node*& head, Node* new_node);
void push_back(Node*& head, Node* new_node);

Node* pop_front(Node*& head);
Node* pop_back(Node*& head);

void clear(Node*& head);

/*****
// additional functions not in assignment 8
*****/

// return pointer to the node at position
// if position is off the end of the list, return nullptr
Node* get_node_at(Node* head, size_t position);

// insert new_node at the given position, and return the pointer to that node
// if position is out of bounds, return nullptr
Node* insert_at(Node*& head, Node* new_node, size_t position);

// remove the node at the given position from the list
// erase_at DOES NOT release the memory of that node
// instead it returns a pointer to that node
// if position is out-of-bounds, return nullptr
Node* erase_at(Node*& head, size_t position);
```

Notes

- These functions manage **links only**
- Node allocation occurs in **Sequence**
- Returned nodes must be deleted by the caller (**Sequence**)
- You will want to **test** all the functions in `linked_list.h` **before** you move on to implementing **Sequence**

The Sequence Class

Files

Your implementation must be split across the following files:

- `Sequence.h`
- `Sequence.cpp`

`Sequence.h` must contain the *declaration* for `Sequence`. In addition to the required methods shown below, you will need to put `ElementType` in the `public` section.

Forward Declaration of `Node`

You will need to add what is called a *forward declaration* for the `Node` type. A forward declaration tells the compiler that a type exists, without providing its full definition. This allows you to declare pointers to that type before it is fully defined. In this project, you must forward-declare `Node` **before** declaring the `head` pointer in the `private` section of `Sequence`.

```
// Sequence.h

#pragma once
#include <string>

class Sequence {
public:
    using ElementType = std::string;

    Sequence();
    ~Sequence();

    void push_back(const ElementType& value);
    // ...
private:
    struct Node; // forward declaration
    Node* head;
};
```

The full definition of `Node` appears in `linked_list.h`. Because `Sequence` only stores a pointer to a `Node`, the compiler does not need to know the complete structure at this point.

This separation is intentional and helps keep implementation details hidden.

Including `linked_list.h` in `Sequence.cpp`

You must `#include "linked_list.h"` **only** in `Sequence.cpp`.

It should **not** be included in `Sequence.h`.

Because the linked list implementation depends on the element type use by `Sequence`, you must define an alias for `ElementType` *before* including `linked_list.h`.

The `Sequence` class defines its element type as:

```
using ElementType = std::string;
```

However, inside `linked_list.h`, the `Node` struct refers to `ElementType` directly and does not know about the `Sequence` class. To resolve this, you must create a local alias in `Sequence.cpp` that connects the two.

For example:

```
// Sequence.cpp

#include "Sequence.h"

using ElementType = Sequence::ElementType;
#include "linked_list.h"
```

This alias tells the compiler that `ElementType` used by the linked list refers to `Sequence::ElementType`. This approach allows the linked list implementation to remain value-agnostic while letting `Sequence` control the element type.

This separation is intentional and mirrors how internal helper modules are wired together in real C++ systems.

Required Public Interface

Your `Sequence` class **must** provide the following public interface.

Method signatures must match exactly.

Constructors/Destructor

`Sequence()`

Default constructor. Initializes the Sequence to an empty state with no elements.

`~Sequence()`

Destructor. Removes all elements from the Sequence and releases all dynamically allocated memory.

Insertion Operations

`void push_front(const ElementType& value)`

Inserts a new element at the beginning of the Sequence. The new element becomes the first element.

`void push_back(const ElementType& value)`

Inserts a new element at the end of the Sequence. The new element becomes the last element.

`void insert(size_t position, const ElementType& value)`

Inserts a new element at the specified position. Elements at and after this position are shifted to the right.

Valid positions satisfy: $0 \leq \text{position} \leq \text{size}()$.

Removal Operations

`ElementType pop_front()`

Removes the first element from the Sequence and returns it. Throws an exception if the Sequence is empty.

`ElementType pop_back()`

Removes the last element from the Sequence and returns it. Throws an exception if the Sequence is empty.

`ElementType erase(size_t position)`

Removes the element at the specified position and returns it.

Valid positions satisfy: $0 \leq \text{position} < \text{size}()$. Throws an exception if the position is invalid.

Access Operations

`const ElementType& front() const`

Returns a constant reference to the first element in the Sequence. Throws an exception if the Sequence is empty.

`const ElementType& back() const`

Returns a constant reference to the last element in the Sequence. Throws an exception if the Sequence is empty.

```
const ElementType& get(size_t position) const
```

Returns a constant reference to the element at the specified position.

Valid positions satisfy: $0 \leq \text{position} < \text{size}()$. Throws an exception if the position is invalid.

Cardinality Operations

```
bool empty() const
```

Returns `true` if the Sequence contains no elements; otherwise returns `false`.

```
size_t size() const
```

Returns the number of elements currently stored in the Sequence.

Utility Operations

```
void clear()
```

Removes all elements from the Sequence and releases all associated memory. After this call, the Sequence behaves as if newly constructed.

Output Operator

```
friend std::ostream& operator<<(std::ostream& os, const Sequence& s)
```

Outputs all elements of the Sequence to the provided output stream in order. Exact formatting is specified in the test harness.

Design Requirements

Sequence must:

- Allocate nodes using `new`
- delete nodes when removed
- delete all nodes in its destructor

Sequence must **not**:

- expose nodes publicly
- manipulate `next` pointers directly
- reimplement linked list mechanics

Error Handling

Your implementation must throw a C++ exception when an operation is invalid, including:

- Removing elements from an empty Sequence
- Accessing elements in an empty Sequence
- Using an invalid position for `insert`, `erase`, or `get`

You may throw a generic exception type, the specific exception class is not graded.

Output Operator

The output operator must display all elements in order.

Traversal logic may:

- use node-level helpers, or
- use `Sequence` accessors

Both approaches are acceptable as long as behavior is correct.

Additionally, the tests I wrote require the output to surround the elements with `<>`. So if the `Sequence` contains the letters A, B, C, D, and E, you should format your output like: `<A, B, C, D, E>`. You will need to have a conditional to determine whether to output the comma and space after everything except the last element.

Formatting the output is not vital to the assignment, however the output operator tests will fail. As long as you output the `Sequence`, failing that test doesn't mean you will lose points.

Implementation Requirements

- You must **not** use STL containers internally
 - All dynamic memory must be managed correctly.
 - Your implementation must not leak memory.
 - you must not `#include .cpp` files
-

Test Harness

A test harness will be provided and used for grading.

Before running the test harness, you should write your own tests. I have provided a separate file, `SequenceDebug.cpp` that can be compiled and run separately from the test harness.

Your code must match the specified interface in order to compile and pass the tests.

Documentation and Style

CS 3100 is the capstone programming course of your academic career. You should employ professional best practices in your coding and documentation. These practices include:

- **Descriptive variable names** — Avoid variable names like `p` and `temp`. Use names that describe the purpose of the variable. Use either underscores or camelCase for multiple-word names. Examples of appropriate variable names include: `currentNode`, `toBeDeleted`, and `studentList`. Exception: It is generally acceptable to use single-letter variables, such as `i`, as loop indices in for loops.
- **Function header comments** — At a minimum should include the name of the function, its purpose, a list of input parameters, the return value, and any side effects.
- **Whitespace and inline comments** — Code should be broken into blocks of 5 to 20 lines of code, separated by whitespace, and annotated with inline comments. Any particularly hard to understand lines of code should have an explanatory inline comment.

IMPORTANT NOTES

You should make a habit of committing your code regularly. If your repository shows a few large commits and few others, that will flag your submission for potential academic integrity violation.

The course policy disallows A.I. on ANY assignment. Any A.I. use will result in a zero on the assignment and an academic integrity violation.

Programming assignments for this course are INDIVIDUAL. Submissions are partially or fully copied from another submission will receive a zero and an academic integrity violation

Code that will not compile will receive a grade of zero. Also note, if you `#include` a `.cpp` file, your code will not compile in our tests. Only `#include .h` files.

Turn in and Grading

- Commit `Sequence.h`, `Sequence.cpp`, `linked_list.h`, and whatever tests you wrote in `SequenceDebug.cpp` to your GitHub Classroom repository.
- Push your changes to the remote repository.
- The most recent commit on the remote repository will be graded.

This project is worth **50 points**, distributed as follows:

Description	Points
Insertion operations	
<code>push_front()</code> (handles empty and non-empty cases)	5
<code>push_back()</code> (handles empty and non-empty cases)	5
<code>insert(position)</code> at arbitrary positions	4
Removal operations	
<code>pop_front()</code> (correct unlinking and return value)	5
<code>pop_back()</code> (correct unlinking and return value)	5
<code>erase(position)</code> with correct node deallocation	4
Access operations and state queries	
<code>front()</code> , <code>back()</code> , and <code>get(position)</code>	5
<code>empty()</code> and <code>size()</code>	3
Memory management	
Destructor correctly releases all nodes	5
<code>clear()</code> removes all elements without leaks	5
Code quality	
Code conforms to CS 3100 Coding Standards	4
Total	50